

AI 활용 기술서

화월 : Nylon Pong

Git 기반 Workspace를 Project OS로 구성한 사람·AI 협업 기록

허승 · 팀원 B · 팀원 C

2026년 8월

목차

목차를 클릭하면 해당 장으로 이동합니다. 페이지 번호가 비어 있으면 목차를 선택하고 F9 를 눌러 갱신하세요.

읽는 순서 5

1. AI 활용 배경과 문제 정의 5

1.1 세 사람이 각자 AI를 쓰면 생기는 일 6

1.2 그래서 만든 것 6

2. AI 협업 시스템 설계 목표 6

3. 전체 AI 협업 아키텍처 7

3.1 Workspace는 코드를 저장하는 곳이 아니다 8

4. Git 기반 Workspace(Project OS) 기술 구조 9

4.1 실제 디렉터리 구성 9

4.2 AGENTS.md — AI 공통 진입 규칙 9

4.3 정보의 정본과 이중 정본 금지 10

5. 자연어 Intake → Task → AI 작업 → Human Review 11

5.1 자연어 업무 요청 Intake 11

5.2 Task 상태와 전이 권한 11

5.3 작업 시작 게이트 12

5.4 담당자와 실제 작업자의 분리 12

5.5 AI 입력·출력과 사람 관문 12

6. AI 대상 주요 Prompt와 공통 지시사항 13

6.1 상시 적용되는 공통 지시 13

6.2 작업 유형별 지시 구조 14

6.3 Quick Guidance — "지금 뭐 하면 돼?" 14

6.4 사람이 실제로 준 지시 15

7. 서버·디자인·화면에서의 실제 활용 사례 15

7.1 서버 — P2-02 게임 상태 머신과 Redis 16

7.2 디자인 — PROD-004 시각 에셋 제작 16

7.3 화면 — PROD-017 유도형 튜토리얼 17

8. 3인 협업과 Repository 확장 구조 17

8.1 저장소별 소유 정본과 책임 경계 17

8.2 Git만으로 성립하는 협업 18

8.3 Repository Registry와 확장 19

9. AI 활용 효과와 실제 작업 추적 19

9.1 이 구조로 처리한 작업 19

9.2 Task 하나를 처음부터 끝까지 — P2-02 21

9.3 사람의 검토로 결과가 바뀐 사례 21

9.4 이 구조가 만든 차이 22

10. Human-in-the-loop와 AI 사용 범위 23

10.1 AI가 한 일과 사람이 한 일 23

10.2 AI가 결정하지 않는 것 23

10.3 사람이 수행한 품질 검토 23

11. 지금의 범위와 다음 단계 24

11.1 지금은 사람이 맡고 있는 것 24

11.2 같은 구조 위에서 이어 붙일 수 있는 것 24

12. 사용 AI·외부 에셋·오픈소스와 라이선스 25

12.1 사용한 AI 도구 25

12.2 AI로 생성한 에셋과 후처리 26

12.3 외부 에셋 출처와 라이선스 26

12.4 오픈소스와 제작·검수 도구 27

12.5 민감정보와 저작권을 다룬 방식 28

부록. 기준 revision 28

화월 : Nylon Pong — 나이롱뽕 온라인 멀티플레이 게임

항목	내용
팀 구성	3명 — 허승(화면), 팀원 B(서버), 팀원 C(기반·디자인)
개발 기간	2026-07-18 ~ 2026-08-07
저장소	`workspace`, `fe`, `be`, `design`, `infra` (GitHub Organization `GameProject-Nylon-ppong`)
서비스 주소	https://playhwawol.kr
다루는 내용	AI 협업 구조의 설계와 실제 적용, 주요 지시사항, 외부 에셋·오픈소스 출처

> 이 프로젝트는 생성형 AI에 코드와 에셋을 요청해 받아 쓰는 방식 대신, **Git 저장소를 Project OS로 구성해 세 사람이 각자의 AI Agent를 쓰면서도 같은 프로젝트 문맥과 Task 체계를 공유하도록 설계**하고, 그 구조 위에서 게임을 만들었다.

읽는 순서

- 1~2장: 왜 이 구조를 만들었는지와 설계 목표
- 3~5장: 전체 아키텍처, Workspace 기술 구조, 실제 작업 프로세스
- 6~7장: AI에게 준 공통 지시와 세 축의 실제 활용 사례
- 8~10장: 3인 협업·확장 구조, 정량 자료와 작업 추적, 사람이 맡은 판단
- 11~12장: 지금의 범위와 다음 단계, 사용 AI·외부 에셋·오픈소스 출처

◆

1. AI 활용 배경과 문제 정의

이 프로젝트에서 AI는 코드와 에셋을 받아 오는 창구가 아니라, **사람이 정한 범위 안에서 일하는 팀원**이었다. 그렇게 만들기 위해 먼저 협업 구조를 설계했다.

1.1 세 사람이 각자 AI를 쓰면 생기는 일

이 팀은 세 사람이 각자 AI Agent를 쓰며 게임을 만들었다. 시작하자마자 부딪힌 것은 AI의 성능이 아니라 **문맥이었다.**

- 개발자마다 자기 AI에게 프로젝트를 처음부터 설명해야 해서 프로젝트 문맥이 사람 수만큼 갈라진다.
- 프로젝트가 길어질수록 이전에 내린 결정을 매번 다시 설명해야 한다.
- AI가 현재 기획과 다른 구현을 내놓아도, 그것이 기획과 다른지 판단할 공통 기준이 없다.
- FE·BE·Design 사이의 요청 전달이 사람의 기억과 대화에 의존한다.
- 기획이 바뀌면 어떤 기능과 문서가 영향을 받는지 추적하기 어렵다.

상용 오케스트레이션 플랫폼을 도입하는 방법도 있었지만, 3인 프로젝트에는 비용과 운영 부담이 컸다. 그래서 **이미 쓰고 있는 Git 위에서 해결하기로 했다.**

> **목표** — "3인이 각자의 AI를 쓰더라도 하나의 프로젝트 문맥을 공유하게 하자." 중앙 서버도 데이터베이스도 새로 세우지 않는 것을 조건으로 뒀다.

1.2 그래서 만든 것

Git 저장소 하나를 **Project OS**로 세웠다. 사람이 승인한 업무가 Task로 고정되고, AI는 그 Task가 지정한 문맥만 읽어 산출물을 만들고, 담당자가 검증해 채택하며, 결과가 다음 역할로 인계된다. 이 순환으로 프로젝트 기간 동안 Task 61건을 처리했다.

◆

2. AI 협업 시스템 설계 목표

목표	실제로 정한 규칙	정본
사람 중심 승인	`completed` 전이는 사람만 가능. 승인 전 완료 처리 금지	`WORKFLOW.md`, `conventions/approval-policy.md`
자연어로 업무 요청	자연어 요청은 실행 지시가 아니라 분석 요청으로 접수	`WORKFLOW.md` Intake

목표	실제로 정한 규칙	정본
Task 단위 실행	승인된 Task 범위 밖 파일 변경 금지	`AGENTS.md` 3항
AI·사람 역할 분리	`current_assignee`(책임)와 `actual_worker`(실작업) 필드 분리	`conventions/task-convention.md`
문맥 유지 비용 절감	Task의 `required_context`에 적힌 문서만 읽게 함	`AGENTS.md` 2항
Git 단일 협업 저장소	별도 서버·DB 없이 Git 저장소만으로 상태·이력·승인 처리	`conventions/git-workflow.md`
확장 가능	저장소를 Registry에 등록하는 방식으로 역할 추가	`current/repositories.yaml`

◆

3. 전체 AI 협업 아키텍처

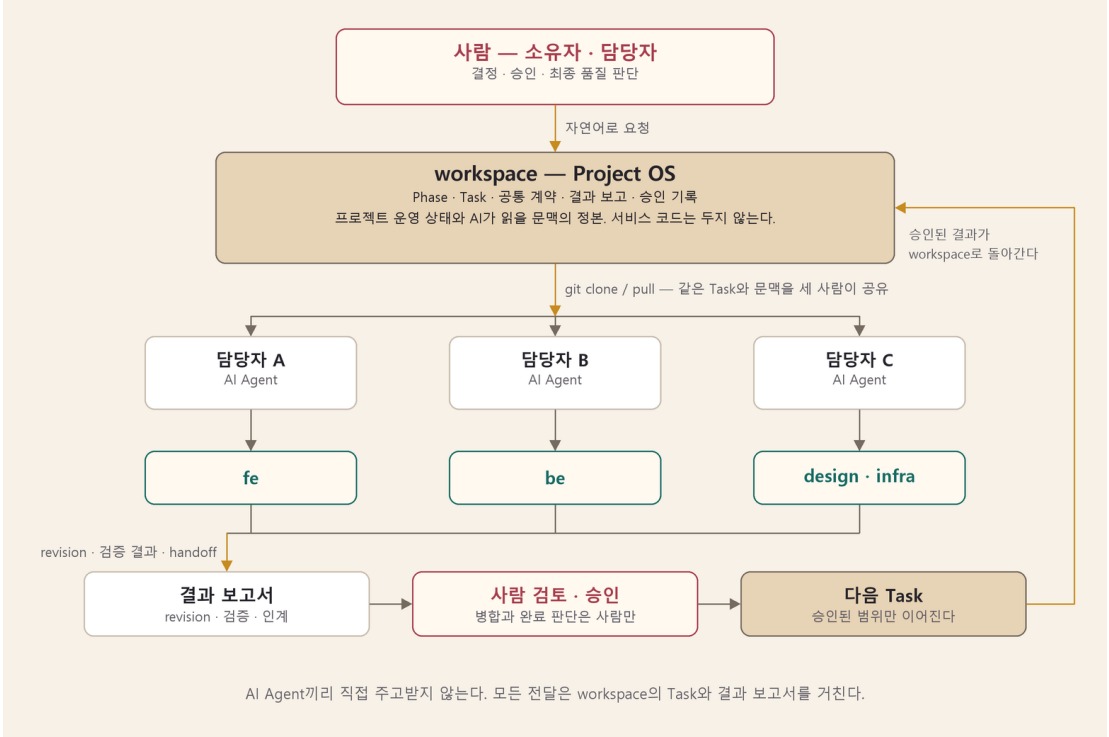


그림 1. Git 기반 AI 협업 아키텍처 — 요청에서 승인까지의 한 사이클

이 그림에서 중요한 것은 화살표의 방향이다. AI Agent는 서로 직접 이야기하지 않는다. 모든 교환은 `workspace`의 Task와 결과 보고서를 거치며, 그 사이마다 사람의 승인 지점이 있다.

3.1 Workspace는 코드를 저장하는 곳이 아니다

저장소	저장하는 것
`workspace`	프로젝트 운영 상태와 AI가 읽을 문맥 — Phase, Task, 계약, 승인, 결과 보고
`fe`	프론트엔드 실제 구현
`be`	백엔드 실제 구현
`design`	디자인 시스템·시각/음향 정본과 FE 전달 명세
`infra`	배포 자동화와 실행 환경

`workspace`에는 게임 서비스 코드가 없다. "지금 무엇을 해야 하는가"에 대한 답이 항상 여기에 있다는 것이 이 저장소의 존재 이유다.

◆

4. Git 기반 Workspace(Project OS) 기술 구조

4.1 실제 디렉터리 구성

```
workspace/
├ AGENTS.md          AI가 진입할 때 따르는 공통 실행 규칙
├ WORKFLOW.md        상태 전이 · Intake · Quick Guidance 절차
├ QUICKSTART.md      사람이 첫 작업을 시작하는 순서
├ current/           현재 상태의 정본
|   ├ active-phase.yaml   지금 해결 중인 Phase와 owner
|   ├ active-tasks/       활성 Task별 포인터 (ID.yaml)
|   ├ project-index.md    진입용 인덱스
|   └ repositories.yaml   저장소 Registry
├ tasks/             Task 정본. 파일 위치가 곧 상태
|   └ draft · backlog · approved · in-progress · review · blocked · completed
├ contracts/         FE·BE 공통 논리 계약 (v1)
├ specs/             게임 규칙 · 시스템 설계 · API · 튜토리얼 명세
├ reports/           Task별 결과 보고서와 증적
├ templates/         Task · 결과 보고 · Intake · Quick Guidance 서식
├ conventions/       Git · 승인 정책 · 컨텍스트 정책 · Task 규약
├ manual/            사람용 절차서와 규칙 검증 예시
└ scripts/           Task 검사 · 계약 검사 · 문서 빌드 스크립트
```

4.2 AGENTS.md — AI 공통 진입 규칙

`AGENTS.md`는 개발자마다 AI에게 긴 프롬프트를 반복 입력하지 않도록, AI가 저장소에 진입할 때 따라야 할 규칙을 8개 항으로 고정한 파일이다. 실제 항목은 다음과 같다.

#	규칙
1	`current/active-phase.yaml`과 할당된 활성 Task를 먼저 확인한다
2	전체 저장소를 탐색하지 말고 Task의 `required_context`만 우선 읽는다
3	승인된 Task 범위 밖의 파일을 변경하지 않는다
4	기술 스택, 사양, 담당자를 추측하지 않는다
5	결과는 지정 보고서에 검증 내용과 `handoff_summary`를 남긴다

#	규칙
6	사람 승인 전 Task를 `completed`로 처리하지 않는다
7	상태 변경 시 문서의 `status`와 Task 디렉터리를 함께 맞춘다
8	자연어 업무 요청은 Task나 코드로 바로 만들지 말고 Intake 절차를 따른다

2항이 문맥 비용을 직접 줄이는 규칙이다. AI는 저장소 전체가 아니라 Task가 지정한 문서만 읽으므로, 같은 배경 설명을 매번 프롬프트에 넣지 않아도 필요한 문맥에 도달한다.

4.3 정보의 정본과 이중 정본 금지

상태를 두 곳에 적으면 곧 어긋난다. 그래서 정본의 위치를 규칙으로 못 박았다.

정보	정본
Task 상세	`tasks/<status>/...md` — 파일의 `status`와 디렉터리 이름은 항상 같아야 한다
활성 Task 포인터	`current/active-tasks/<TASK-ID>.yaml`의 `path` 필드
현재 Phase	`current/active-phase.yaml`
결과	Task가 지정한 `reports/...md`
저장소 등록부	`current/repositories.yaml` — 각 저장소의 역할·쓰기 가능한 Task 유형·지침
전체 요약	위 정본에서 생성하는 파생 정보이며 직접 편집 원본으로 쓰지 않는다

상태 전이는 파일 이동과 문서 `status` 변경, 활성 포인터 갱신을 한 변경에서 함께 처리한다.

◆

5. 자연어 Intake → Task → AI 작업 → Human Review

5.1 자연어 업무 요청 Intake

자연어 요청은 실행 지시가 아니라 **분석 요청**으로 접수한다. 분석 중에는 Task, 변경 요청 파일, 코드, 활성 포인터를 만들지도 고치지도 않는다.

```
사람의 자연어 요청
        |
        v
1. 요청 정리          목적과 관찰 가능한 기대 결과를 다시 쓴다
        |
        v
2. 제한된 현황 확인    project-index → 관련 Feature/Task만 추가 열람
        |                  (전체 저장소 탐색은 기본 동작이 아니다)
        v
3. 중복·영향 분석      중복 / 부분 중복 / 관련 없음 / 판단 자료 없음
        |             FE·BE·Design·Infra·AI·Data·Tools·Workspace 영역별
        |             필수 / 가능성 있음 / 영향 없음 / 확인 필요
        v
4. 질문 또는 후보 제안   목적·범위·제외 범위·완료 조건·대상 저장소·선행 조건
        |
        v
5. 두 번의 승인 경계
        |
        +--- (1) "이 후보로 Task 초안을 생성할까요?"  → 승인 시 tasks/draft/ 에 초안만
        |
        +--- (2) Task 실행과 활성 포인터 등록은 별도의 명시적 승인
```

`draft` 생성 승인은 실행 승인이 아니다. 초안은 `tasks/draft/`에만 만들고 `current/active-tasks/`에 등록하지 않는다.

5.2 Task 상태와 전이 권한

```
draft → backlog → approved → in-progress → review → completed
                          ↘ blocked ↗
```

전이	필수 근거	결정 주체
`backlog` → `approved`	범위·제외 범위·완료 조건·대상 저장소 승인	사람
`approved` → `in-progress`	현재 담당자의 실행 시작 기록 + 활성 포인터	담당자

전이	필수 근거	결정 주체
`in-progress` → `review`	산출물·결과 보고·검증 근거 준비	작업자 보고, 사람 검토 대기
`review` → `completed`	결과 검토와 명시적 승인	사람만 가능

5.3 작업 시작 게이트

새 Task 브랜치를 만들기 전에 순서대로 확인한다. 하나라도 확인되지 않으면 브랜치를 안내하지 않고 사람이 수행할 다음 한 단계만 제시한다.

- 현재 Phase·활성 Task·상태·담당자·대상 저장소
- 대상 저장소의 현재 브랜치, 최신 원격 기준선, tracked 작업 트리
- 이전 Task의 push와 사람에 의한 `dev` 병합, 로컬 `dev`의 최신화
- 브랜치·commit·push·PR·병합은 사람이 명시적으로 요청하기 전에는 실행하지 않는다

5.4 담당자와 실제 작업자의 분리

AI가 작업했다는 사실과 그 결과에 책임지는 사람을 같은 칸에 적지 않는다. Task 정본은 세 필드를 따로 둔다.

필드	뜻	P2-02의 값
`original_assignee`	Task가 처음 정의될 때의 담당	팀원 B
`current_assignee`	현재 책임을 지고 검토·인계를 받는 담당	팀원 B
`actual_worker`	실제 산출물을 만든 작업자	`codex`

AI가 `actual_worker`로 기록돼도 `current_assignee`와 project-owner의 검토·승인 책임을 대신하지 않는다.

5.5 AI 입력·출력과 사람 관문

단계마다 AI에게 허용된 입력, AI가 내는 출력, 통과해야 하는 관문을 나눠 뒀다.

단계	AI에 허용되는 입력	출력	사람·자동 관문
범위 설정	승인 Task·완료 조건·제외 범위	실행 계획·확인 질문	project-owner 승인

단계	AI에 허용되는 입력	출력	사람·자동 관문
근거 수집	`required_context`·등록 저장소 파일·Git 이력	확인 사실·불확실성	경로·revision 대조
생성·수정	계약·명세·승인 참조·현재 코드	코드·문서·에셋 후보	lint/test/render/QC
반영	승인 후보·수정 지시	정본 파일·런타임 연결	담당자 실측·청취·브라우저 검수
종료	검증 결과·남은 제한	결과 보고·`handoff_summary`	사람 `review/completed` 승인

◆

6. AI 대상 주요 Prompt와 공통 지시사항

이 프로젝트에서 프롬프트는 대화창에 매번 입력하는 것이 아니라 **저장소 파일에 고정돼 있다.** 사람이 짧은 자연어로 요청하면 문맥은 Task가 공급한다.

6.1 상시 적용되는 공통 지시

세 사람의 AI가 공통으로 따르는 지시다. 프로젝트 배경을 매번 설명하지 않아도 되는 이유가 여기에 있다.

지시	위치	효과
진입 순서와 금지 사항 8항	`AGENTS.md`	프로젝트 배경 재설명 불필요
Task별 읽을 문서 지정	각 Task의 `required_context`	전체 탐색 없이 필요한 문맥에 도달
상태 전이·승인 절차	`WORKFLOW.md`	완료 판정을 AI가 하지 않음
브랜치·commit·PR 규약	`conventions/git-workflow.md`	Git 동작을 사람 승인 뒤에만 수행
결과 보고 서식	`templates/work-result.md`	revision·검증·제한·인계를 빠뜨리지 않음

6.2 작업 유형별 지시 구조

작업 성격에 따라 AI에게 주는 입력과 요구하는 출력, 통과해야 할 검토가 달랐다.

작업	목적·핵심 입력	요구 출력	검토·최종 반영
구현·문서	Task 정본, `required_context`, 현재 revision, 계약	범위 안의 코드·문서·검증 보고	lint/test/render 후 PR·사람 승인
ImageGen	승인 참조, identity, 16:10 safe area, 금지 요소	문자·UI·워터마크 없는 PNG 후보	후처리·QC·후보 승인 뒤 Design/FE
튜토리얼	규칙, realtime 계약, BE 전이, FE `ActionDock`	화자·대사·`expectedAction` 구조	2회 사람 검수, 계약 불일치 수정
BGM 선택	팀이 직접 내려받은 후보, 제작자·AI 표시·라이선스 정보	역할별 후보 배치와 catalog 연결	사람 청취 후 3곡 승인

6.3 Quick Guidance — "지금 뭐 하면 돼?"

사람이 다음 행동을 물으면 AI는 전체 현황을 늘어놓지 않고 **한 가지 행동**을 3~6줄로 답한다.

비개발자도 같은 질문으로 현재 상태를 확인할 수 있게 하려는 규칙이다.

```
사람: "지금 뭐 하면 돼?"
        |
        v
1. current/project-index.md
2. current/active-phase.yaml
3. current/active-tasks/ 의 Task별 포인터
4. 현재 사용자에게 배정된 Task
5. 필요한 경우에만 해당 Task 정본과 그 Task가 참조한 문서
6. Git 질문인 경우에만 해당 저장소의 현재 Git 상태
        |
        |  (필요한 답을 얻는 즉시 확인을 멈춘다)
        v
한 행동 선택 — 처음 충족하는 항목 하나
  1) 배정된 in-progress 계속   2) review 검토   3) 선행 충족된 approved 시작
  4) draft/backlog 승인 요청   5) 다음 Task 후보 초안 제안   6) blocked 해제 행동
```

응답 형식은 `templates/quick-guidance.md`로 고정돼 있다.

```
지금 할 일: <TASK-ID 제목 또는 한 가지 행동>
저장소: <확인된 대상 저장소>
브랜치: <필요하고 확인된 경우에만>
막힘: <승인 또는 선행 조건이 있는 경우에만>

<사용자가 바로 입력할 다음 명령 또는 자연어 요청>
```

승인되지 않은 Task 실행은 추천하지 않는다. 다음 Task 후보가 문서에 없으면 ID·제목·범위를 만들어내지 않고 "다음 Task 후보 정의"를 제안한다. 상황별 응답이 이 규칙을 벗어나지 않도록 `manual/`에 검증 시나리오를 두었다.

6.4 사람이 실제로 준 지시

개발 요청은 "이 문서 읽고 이렇게 구현해줘" 같은 긴 프롬프트가 아니었다. 대부분 **결과물의 문제를 짚는 짧은 문장**이었고, 무엇을 어떻게 고쳐야 하는지는 AI가 Task와 계약을 다시 읽어 스스로 찾았다.

실제 지시	대상	결과
청단 motif 후보의 우측 여백이 부족하다	PROD-004 (시각)	후보를 정본으로 올리지 않고 재제작
10부족 motif의 고리 조각이 8개다	PROD-004 (시각)	규칙대로 9개가 되도록 재제작
카드 타격음 후보가 기대한 질감이 아니다	PROD-013 (사운드)	청취 후 v1 반려 → v1.1 → v1.2까지 재제작
튜토리얼 기획이 인게임 플로우를 이해하지 못하고 있다	PROD-017 (튜토리얼)	계약·규칙 정본 대조로 대본 구조 3건 수정

대화 전문은 보관하지 않는다. 개인정보·계정 식별자·토큰·비밀값이 기록에 흘러들지 않게 하려는 선택이며, 대신 목적·입력·출력·검토 관계를 Task와 결과 보고서에 남긴다.

◆

7. 서버·디자인·화면에서의 실제 활용 사례

세 사람은 각자 다른 축을 맡았고 AI를 쓰는 방식도 달랐다. 그러나 순서는 같았다 — **후보를 만드는 것은 AI, 정본으로 올릴지 정하는 것은 사람.**

사례	축	담당	실제 작업자
P2-02 게임 상태 머신과 Redis	서버	팀원 B	`codex`
PROD-004 시각 에셋 제작	디자인	팀원 C	팀원 C + Codex
PROD-017 유도형 튜토리얼	화면	허승	`claude`

7.1 서버 — P2-02 게임 상태 머신과 Redis

계약 `contracts/v1`의 `realtime.yaml`·`openapi.yaml`·`errors.yaml`을 `required_context`로 주고, 그 계약이 정의한 phase를 따르는 게임 Aggregate와 Redis 원자 처리·타이머·복구 로직을 구현하게 했다.

AI가 만든 것은 코드였지만, 이 작업에서 중요한 것은 **계약을 먼저 읽게 한 구조**였다. 서버가 게임 규칙의 판정 권한을 갖기 때문에 계약과 어긋난 구현은 화면 전체를 어긋나게 한다. 결과는 전체 테스트 104건, `bootJar`, 계약 v1 공식 검증(4 examples, 48 cards)을 모두 통과했고, 담당자 팀원 B가 확인한 뒤 `completed`로 승인했다. commit·push·PR·병합은 사람이 직접 했다.

7.2 디자인 — PROD-004 시각 에셋 제작

Codex 내장 ImageGen으로 카드 원화, 역 emblem·motif, 캐릭터, 공용 이펙트 후보를 만들었다. 입력 조건은 승인된 참조 이미지, 캐릭터 identity, 전통 소재와 팔레트, 16:10 safe area, 그리고 **문자·UI·워터마크를 넣지 말 것**이었다. 생성 뒤에는 로컬 chroma 제거와 Pillow 후처리를 거쳤다.

이 축에서 사람의 판단이 가장 자주 개입했다. 역 문양은 개수 자체가 규칙이라 — 오광은 광휘 5개, 10부족은 고리 조각 9개 — 조금만 달라도 뜻이 틀어진다. 조각이 8개로 나온 후보, 여백이 부족한 후보, 낱장 꽃잎 3개를 완성된 꽃 3송이로 해석한 후보는 정본으로 올리지 않고 다시 만들었다. 채택하지 않은 후보도 증적으로 남겨 무엇을 왜 골랐는지 추적할 수 있게 했다.

> 이 사례는 'AI 이미지 생성 → 바로 제품 반영'이 아니다. `후보 → 후처리·검수 → 사람 승인 → Design 정본 → FE 연결`이라는 저장소 간 인계다.

7.3 화면 — PROD-017 유도형 튜토리얼

게임 규칙, `realtime` 계약, BE 상태 전이, FE `ActionDock`, 기존 콘텐츠를 읽고 대사 구조와 화면 후보를 만들게 했다. 담당자가 두 차례 검수해 대본 구조와 표현을 다듬었다.

이 작업에서 구조가 가장 잘 드러난 순간은 계약에 없는 역 코드를 발견했을 때다. 공통 계약은 임의로 고칠 수 없게 되어 있어 **작업을 멈추고 보고했고**, 소유자가 범위 확장을 승인한 뒤에야 계약과 콘텐츠, 검증기를 함께 정리했다. 규칙이 없었다면 계약이 조용히 바뀌었을 자리다.

◆

8. 3인 협업과 Repository 확장 구조

8.1 저장소별 소유 정본과 책임 경계

저장소	소유 정본	하지 않는 일	대표 인계
`workspace`	Phase·Task·승인·계약·결과 보고	FE/BE 구현 코드 소유	승인 범위·계약·revision을 각 역할에 전달
`design`	토큰·시각/모션 명세·원본 에셋·FE 전달 규칙	FE 코드·배포 구성 변경	승인 master·manifest·playback spec → FE
`fe`	브라우저 UI·상태·상호작용·런타임 에셋	서버 데이터·원본 에셋 권리 정본	화면 실측·통합 검증 결과 → Workspace
`be`	서버 실행·데이터·API·실시간 상태 전이	FE 화면·Design 원본 변경	계약·테스트·복구 동작 → FE/Workspace
`infra`	배포 자동화·실행 환경·CI/CD·복구 절차	애플리케이션 기능·디자인 산출물 변경	환경·배포·운영 검증 → 각 구현 역할

저장소가 나뉘어 있어도 한 산출물의 원본·구현·배포 절차가 서로 다른 곳에 있을 수 있다. 그 사이를 Task와 결과 보고서가 잇는다.

- Workspace의 Task가 소비할 계약·`required_context`·`target_repositories`를 고정한다.

- Design 또는 BE가 자기 저장소 정본과 검증 가능한 revision을 만든다.
- FE는 전달받은 master·manifest·계약을 실제 런타임과 화면에 연결하고 실측한다.
- 결과 보고서가 사용한 revision, 반영 경로, 남은 제한을 `handoff_summary`로 Workspace에 반환한다.
- 다음 역할은 다른 저장소 코드를 추측하지 않고 Task·결과 보고·확인된 revision만 소비한다.

> **범위 확대 금지** — 한 작업 중 다른 저장소 수정이 필요해져도 편의상 함께 고치지 않는다. 대상 저장소가 Task에 없으면 별도 영향 분석과 승인을 받아야 하며, 미등록 저장소는 존재한다고 가정하지 않는다.

8.2 Git만으로 성립하는 협업

```
workspace를 git pull
        |
        v
자기 Task 확인 (current/active-tasks/ + Quick Guidance)
        |
        v
각자의 AI Agent로 대상 저장소에서 작업
        |
        v
검증 (lint · test · build · 브라우저 실측 · 청취 QC)
        |
        v
결과 보고서 작성 → 사람이 diff 확인 → commit · push · PR
        |
        v
사람이 dev 병합 → workspace 상태·보고서 갱신 → 다음 Task
```

이 구조는 중앙 오케스트레이션 서버나 별도 데이터베이스를 세우지 않는다. Git 저장소 하나가 **문맥 저장소 · 버전 관리 · 협업 채널 · 변경 이력**을 동시에 맡는다.

Git을 선택해 얻은 것	근거
별도 DB·서버 불필요	운영 상태가 전부 저장소 안의 파일
변경 이력 추적	Task ID를 브랜치·commit·PR·결과 보고서에 연결
Branch/PR 기반 사람 승인	`dev` 병합을 사람이 수행하는 것이 승인 지점
같은 문맥 공유	세 사람이 같은 `workspace`를 pull
로컬 AI Agent 연동 용이	AI가 파일 시스템으로 바로 읽음, 별도 API 불필요

8.3 Repository Registry와 확장

저장소는 코드로 하드코딩하지 않고 `current/repositories.yaml`에 등록한다. 각 항목은 `role`, `default_branch`, `writable_task_types`, `agent_instruction_file`을 가진다.

저장소	역할	쓰기 가능한 Task 유형
`workspace`	프로젝트 운영, 기획, Task와 승인 기록의 정본	`planning`, `operations`, `documentation`
`fe`	프론트엔드 서비스 구현	`frontend-development`
`be`	백엔드 서비스 구현	`backend-development`
`design`	디자인 시스템, 시각 산출물과 FE 전달 명세의 정본	`design`
`infra`	배포 자동화, 실행 환경 구성과 운영 절차의 정본	`infrastructure-development`

프로젝트는 `workspace`·`fe`·`be` 세 저장소로 시작해 `design`과 `infra`를 같은 방식으로 추가했다. Registry 파일에는 `ai`, `data`, `tools`를 실제 저장소가 생길 때 같은 필드로 추가하라는 주석이 남아 있다. **역할이 늘어나면 구조를 바꾸는 대신 항목을 추가한다.**

◆

9. AI 활용 효과와 실제 작업 추적

9.1 이 구조로 처리한 작업

프로젝트 기간 동안 **Task 61건**이 이 체계를 거쳤고 그중 **48건이 완료**됐다. Task마다 결과 보고서가 남아 현재 **57건**이 저장소에 있다.

상태	건수
`completed`	48
`in-progress`	5
`review`	3
`backlog`	3

상태	건수
draft	2
합계	**61**
Task 유형	**건수**
frontend-development	19
backend-development	16
planning	6
development	5
documentation	5
operations	4
infrastructure-development	2
design	2
full-stack-development	1
contract-correction	1
대상 저장소 (한 Task가 여러 저장소를 가질 수 있음)	**건수**
fe	27
workspace	24
be	22
design	8
infra	5
actual_worker 기록	**건수**
codex	28
claude	17
사람 단독 (팀원 C 4, 팀원 B 1)	5

상태	건수
사람 + Codex 공동	2
미기재	9

61건 중 **45건에서 AI가 실제 작업자**였고, 그 45건 모두 담당자의 검토를 거쳐 상태가 바뀌었다. AI가 혼자 완료로 넘긴 Task는 하나도 없다.

9.2 Task 하나를 처음부터 끝까지 — P2-02

게임 상태 머신은 서버가 규칙을 판정하는 핵심이라 검증 기준이 가장 엄격했던 작업이다. 요청부터 완료까지 실제로 이렇게 흘렀다.

```
선행 Task P1-02 · P2-01 완료 확인
   v
P2-02「게임 상태 머신과 Redis」Task 정본   (backend-development / 대상 be)
   required_context = AGENTS.md · be/AGENTS.md · WORKFLOW.md · git-workflow.md
                      contracts/v1  realtime.yaml · openapi.yaml · errors.yaml
   v
사람이 실행 승인 — 담당 팀원 B / actual_worker codex
   v
be dev @ bfa4ef3  ->  브랜치 task/P2-02-game-state-machine-redis
   v
구현 — 계약 v1 phase를 따르는 게임 Aggregate, Redis 원자 처리·타이머·재기동 복구
   v
검증 — 전체 테스트 104건 통과 · bootJar 성공 · 계약 v1 검증 통과(4 examples, 48 cards)
   v
결과 보고서 reports/development/P2-02-result.md  (revision·검증·제한·인계)
   v
사람이 commit · push · PR · 병합 수행        <- AI는 Git 동작을 하지 않는다
   v
팀원 B 승인(2026-07-23)  ->  status: completed  /  be 7e549b2
```

AI가 스스로 한 것은 계약을 읽고 구현하고 검증을 돌린 데까지다. 브랜치를 만들고 병합하고 완료로 넘기는 일은 전부 사람이 했다.

9.3 사람의 검토로 결과가 바뀐 사례

AI가 낸 결과가 그대로 정본이 된 적은 없다. 축마다 검토 방식이 달랐다.

사례	축	사람이 짚은 것	이어진 조치
PROD-004 역 문양	디자인	10부족 고리 조각이 규칙과 달리 8개	정본으로 올리지 않고 재제작

사례	축	사람이 짚은 것	이어진 조치
PROD-004 띠 카드	디자인	2월·5월 원화의 띠 경계가 어색함	v3·v4 후보를 다시 만들어 해소
PROD-013 카드 타격음	음향	청취 결과 질감이 기대와 다름	v1 반려 → v1.1 → v1.2까지 재제작
PROD-017 튜토리얼 대본	화면	실제 인게임 조작과 다른 설명	계약·규칙 대조 후 구조 3건 수정
DOC-002 게임 가이드	문서	구현되지 않은 조작을 사실처럼 서술	실제 동작 확인 후 3건 수정

9.4 이 구조가 만든 차이

같은 문맥에서 일한다. 세 사람이 서로 다른 AI Agent를 썼지만 `git pull` 하나로 같은 Phase, 같은 Task, 같은 계약을 본다. 누가 무엇을 하고 있는지 물어보지 않아도 저장소에 적혀 있다.

배경 설명을 반복하지 않는다. 새 작업을 시작할 때 프로젝트를 처음부터 설명하는 대신 "승인된 작업을 진행해줘"로 시작한다. `AGENTS.md`가 진입 순서를, Task의 `required_context`가 읽을 문서를 이미 정해 두었기 때문이다.

변경을 되짚을 수 있다. Task ID 하나로 브랜치·commit·PR·결과 보고서가 이어진다. 반년 뒤에 "이 코드가 왜 이렇게 됐나"를 물으면 그 Task의 승인 근거와 검증 결과까지 따라갈 수 있다.

역할이 늘어나도 절차는 그대로다. `workspace`·`fe`·`be`로 시작해 `design`과 `infra`를 추가했지만 Task 체계는 바꾸지 않았다. Registry에 항목을 더한 것이 전부다.

코드를 안 보는 사람도 현재 상태를 안다. "지금 뭐 하면 돼?"라는 한 문장으로 다음 할 일과 대상 저장소, 막힌 조건까지 3~6줄로 돌아온다.

작업이 끊기지 않고 넘어간다. 결과 보고서의 `handoff_summary`가 다음 작업자에게 사용한 revision과 남은 제한을 넘긴다. 튜토리얼 작업이 선행 Task 두 건을 이어받아 시작할 수 있었던 것도 이 기록 덕분이다.

◆

10. Human-in-the-loop와 AI 사용 범위

10.1 AI가 한 일과 사람이 한 일

AI가 수행	사람이 수행
요구사항 구조화와 Task 분해 후보 제시	기획 최종 결정
영향 범위 분석	Task 승인과 범위 확정
구현·문서·에셋 후보 생성	변경 승인
자동 검증 실행과 결과 정리	구현 결과 확인과 디자인 판단
결과 보고서 작성	`dev` 병합 승인
다음 작업 후보 안내	최종 품질 판단과 `completed` 전이

> **AI의 판단을 최종 의사결정으로 쓰지 않고, 사람의 승인 지점을 절차상 명시적으로 배치했다.**
>
> 단계별 관문은 5.5절, 책임과 실작업의 분리는 5.4절에 있다.

10.2 AI가 결정하지 않는 것

- **Task 범위 확대와 계약 변경.** 필요해 보이면 멈추고 보고한다(7.3절 사례).
- **정본 승격.** 에셋·문서 후보를 정본 경로로 올리는 것은 담당자 승인 뒤에만 한다.
- **라이선스 판단.** 출처나 이용 조건이 불명확하면 채택하지 않고 확인 대상으로 남긴다.
- **PR 병합과 `completed` 전이.** 브랜치·commit·push·병합은 사람이 직접 수행한다.
- **확인되지 않은 것의 단정.** 다른 저장소의 미확인 구현, 담당자, 승인 사실을 추측하지 않는다.

10.3 사람이 수행한 품질 검토

대상	자동 검증	사람 검토
코드	lint, typecheck, 단위·통합 테스트, production/Storybook build	실제 브라우저 흐름 확인

대상	자동 검증	사람 검토
이미지	해상도·투명도·슬롯 매핑 검사	인물·손·문자·도상 오류, 축소 가독성, 실제 UI 충돌
오디오	sample rate·bit depth·channel·peak·길이 QC	사람 청취, loop·Content ID 위험 판단
문서	구조 검사, 목차 정합성, 잔여 마크다운 검사	제목 계층·표·링크·한글 글꼴·페이지 분리 확인

◆

11. 지금의 범위와 다음 단계

이 구조는 3인 프로젝트가 감당할 수 있는 최소한으로 만들었다. 그래서 지금 잘 도는 부분과 아직 사람 손에 남아 있는 부분이 뚜렷하게 나뉜다.

11.1 지금은 사람이 맡고 있는 것

- **상태 정합.** 파일 위치와 `status`, 활성 포인터를 한 번에 맞추는 규칙이 있고 `scripts/validate-task.py`가 이를 검사한다. 검사를 돌리는 것은 아직 사람이다.
- **검증 실행.** 테스트·빌드·브라우저 확인은 각자 로컬에서 돌린다. 자동으로 도는 파이프라인은 아직 없다.
- **작업 기록의 세밀도.** `actual_worker`는 도구 이름까지만 담는다. 모델과 버전이 함께 들어가면 같은 조건을 재현하기 쉬워진다.

11.2 같은 구조 위에서 이어 붙일 수 있는 것

구조를 바꾸지 않고 연결할 수 있는 방향이다. **아직 연동하지 않은 계획**이다.

방향	이어 붙일 지점
CI/CD 자동 검증	Task 완료 조건에 이미 적혀 있는 검증 명령을 그대로 실행
GitHub Issues 연동	Task ID ↔ Issue 번호 대응

방향	이어 붙일 지점
PR 자동 리뷰	`conventions/git-workflow.md`의 규약을 검사 규칙으로
외부 이슈 트래커·메신저 알림	결과 보고서의 `handoff_summary`
`ai·data·tools` 저장소	`current/repositories.yaml`에 항목 추가

저장소를 늘리는 일은 이미 `design`과 `infra`를 추가하며 한 번 검증했다. 나머지는 Task 체계를 그대로 두고 바깥에 연결하는 작업이다.

◆

12. 사용 AI·외부 에셋·오픈소스와 라이선스

12.1 사용한 AI 도구

도구	활용 영역	역할	Task 기록
Codex	`fe·be·workspace`	Task 분석, 코드 구현, 문서화	`actual_worker` 28건
Claude	`fe·workspace`	화면 구현, 튜토리얼 콘텐츠, 제출 문서	`actual_worker` 17건
Codex 내장 ImageGen	`design`	캐릭터·배경·브랜드·서예 후보 생성	PROD-004
Windows SAPI (`Microsoft Heami Desktop`)	`design`	임시 음성 후보 15종	PROD-013
Git / GitHub	전 저장소	문맥 공유, 버전과 변경 이력, PR 기반 승인	전 저장소

모델과 버전은 Task 기록에 남기지 않았다(11.1절).

12.2 AI로 생성한 에셋과 후처리

에셋	생성 도구	입력 방식	후처리
캐릭터·배경·브랜드·서예 후보	Codex 내장 ImageGen	승인 참조 이미지 + identity + 전통 소재·팔레트 + 16:10 safe area + 문자·UI·워터마크 금지	로컬 chroma 제거, Pillow 후처리, 후보 승인 gate
창포 8종	Codex 내장 ImageGen	단색 chroma-key	로컬 배경 제거·Pillow 후처리 (API 키·CLI fallback 미사용)
임시 음성 15종	Windows SAPI Heami	발화 속도·음높이 조정	내부 청취 전용, 배포 승인 없음

BGM 3곡은 이 표에 없다. 프로젝트가 생성한 것이 아니라 **제작자가 AI 생성 표시와 함께 Pixabay에 올린 음원**을 받아 쓴 것이며, 출처는 12.3절에 있다.

12.3 외부 에셋 출처와 라이선스

구분	이름	사용 위치	라이선스	출처	수정
BGM	The Rising Sun	로비·메인	Pixabay Content License (`AI modified or generated`)	kaazoom · Pixabay 470323	X
BGM	Han River Chill	대기방	Pixabay Content License (`AI Generated`)	u_98673jp944 · Pixabay 436915	X
BGM	Seoul Flow	인게임	Pixabay Content License (`AI Generated`)	u_98673jp944 · Pixabay 420938	X
SFX	836449 cancel	UI 오류·실패	CC0 1.0	Feraly_ · Freesound	O — gain·mono·48 kHz

구분	이름	사용 위치	라이선스	출처	수정
SFX	413203 / 413210	승리·패배	CC0 1.0	joepayne · Freesound	O — 구간 선택·gain·mono·48 kHz
SFX	240777 Dealing Card	비교 후보	CC BY 4.0	f4ngy · Freesound	O — 앞 무음 제거·mono·48 kHz
SFX	789790 / 514086	UI 활성·카드 드로우 pool	CC0 1.0	Freesound	O
SFX	MPooman 화투 효과음 4종	현재 반영	제작자 공개 허가 (증적 확인 중)	제작자 직접 문의	O
SFX	`sfx_emergency`	내부 후보 — 배포본 제외	확인 중	확인 중	—

라이선스 원문: Pixabay Content License · CC0 1.0 · CC BY 4.0

BGM 3곡은 제작자가 Pixabay에 AI 생성 표시와 함께 올린 음원이다. 프로젝트가 생성한 것이 아니며 원본을 가공하지 않고 그대로 쓴다. 세 곡 모두 사람이 직접 들어 보고 반복 재생 품질과 권리 조건을 확인한 뒤 채택했다.

12.4 오픈소스와 제작·검수 도구

범위	확인 버전·도구	주요 라이선스·용도
FE 직접 의존성	React 19.2.7, Router 7.18.1, Query 5.101.2, STOMP.js 7.3.0, Zod 4.4.3, Zustand 5.0.14	MIT 중심, STOMP.js Apache-2.0 · UI/상태/통신/검증
FE 제작·검수	TypeScript 6.0.3, Vite 8.1.5, Vitest 4.1.10, Storybook 10.5.2, Oxlint 1.74.0	Apache-2.0 또는 MIT · build/test/UI 검수/lint
BE	Spring Boot 4.1.0, Testcontainers BOM 2.0.5, Gradle 9.6.1, PostgreSQL/Redis	Apache-2.0·MIT·BSD 계열 · API/데이터/통합 테스트

범위	확인 버전·도구	주요 라이선스·용도
에셋·문서 도구	FFmpeg/ffprobe 8.1.2 full, Pillow, Playwright, python-docx, LibreOffice	오디오 QC·후처리·브라우저 실측·Word 생성·렌더

위 표는 직접 의존성 기준이다. lock 파일이 끌어오는 전이 의존성은 수백 항목이므로 배포 시점에 SBOM으로 전체 라이선스를 확인한다. FFmpeg는 빌드 옵션에 따라 LGPL/GPL 적용이 달라져 실제 바이너리 기준으로 따로 확인한다.

12.5 민감정보와 저작권을 다룬 방식

AI에게 넘기는 문맥과 저장소에 남기는 기록 모두에 같은 기준을 적용했다.

- **AI 입력은 Task가 지정한 범위로 제한했다.** `required_context`에 없는 문서는 읽히지 않으므로 불필요한 데이터가 애초에 문맥에 들어가지 않는다.
- **토큰·API 키·계정 식별자·환경 변수 값은 저장소 기록에 남기지 않았다.**
- **외부 에셋은 제작자·제목·원본 URL·라이선스·가공 내용·적용 위치를 한 묶음으로 관리했다.** `design` 저장소의 라이선스 기록과 catalog가 그 정본이다.
- **출처가 확인되지 않은 항목은 배포본에 넣지 않았다.** `sfx_emergency`가 내부 후보로만 남아 있는 것이 그 예다.

◆

부록. 기준 revision

이 문서가 설명하는 저장소 상태다.

저장소	revision	내용
`workspace`	`34d1d79`	Task 정본, `AGENTS.md`·`WORKFLOW.md`·`conventions/`, 결과 보고서
`fe`	`ed382af`	프론트엔드 구현, `package.json`·`package-lock.json`

저장소	revision	내용
be	5ca98c4	상태 전이·계약·테스트, build.gradle, Gradle wrapper
design	2ab01e0	원본 에셋, manifest, playback spec, 음원 라이선스 기록
infra	8d43918	배포 자동화와 운영 환경

본문 사례의 원자료는 tasks/completed/의 Task 정본과 reports/의 결과 보고서에 있다 — P2-02, PROD-004, PROD-013, PROD-017.